When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

Out of the 18 possible scenarios that arise during the audit process, the original system would prevent the failure in 4 of them. On the other hand, ERMA-GRC prevented all the 18 failures. ERMA-GRC rejects writes that occur out of a session, disallows any users from approving their own documents, stamps the audit line with the signed-in user and not the claimed name, captures an edit entry and reports the database unavailability as an error. Outside of the audit process, it prevents 3 out of the 4 failures, including two simultaneous approvals of the same record. It gets the stored results from the database emulator and not from whatever the system claims. There were two instances where the new design failed, and we document both. While one critical risk indicator showed the appropriate state on the screen, a client connecting to the server could set any state it liked. Also, with the audit key, a person can change the audit trail offline without detection even after restarting.

This Abstract is submitted to LEAD-2027 conference conducted at IIM Shillong during 20-22 January 2027

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable propositions towards a design theory, and a scenario protocol that others can rerun. Buyers and auditors may apply the principles to their checklists. The two design failures define the next three: off-system anchoring of audit trails, computing all metrics on the server and validating the concepts through practitioners' testing.

Keywords

assurance integrity; GRC information systems; design science research; enterprise risk management; tamper-evident audit trail; separation of duties